

VeriGate-FW

Developer Manual

Engineering standards · repository structure · Git workflow · testing · CI/CD · secrets · ML & evaluation discipline · phase-by-phase build guide · master checklist

Version 1.1 · 18 September 2026 · applies to repository `verigate-fw`

0. How to use this manual

This manual is for the developer building VeriGate-FW. It assumes you have read `docs/VeriGate-FW_Project_Guide.pdf` (what the system is). This document says **how we build it**: the rules, the tools, the folder layout, and the order of work. Every rule here is enforced by tooling wherever possible (pre-commit hooks, CI), so "I forgot" is not a failure mode.

The repository skeleton described here **already exists** at `~/Desktop/IOT/verigate-fw/` with all config files, workflows, templates and ADRs in place. Your first task is P0-06 in the checklist: make `bootstrap` run clean.

Section	Read when
1 Engineering principles	Before writing any code
2 Toolchain & setup	Day 1
3 Repository layout	Day 1, and whenever you wonder "where does this go?"
4 Coding standards	Before your first PR in each language
5 Git workflow	Before your first branch
6 Testing strategy	Before your first test
7 CI/CD pipeline	Before your first push; when CI fails
8 Secrets & configuration	Before touching keys or networks
9 Daily development loop	Every day
10 Phase-by-phase build guide	At the start of each phase
11 ML & data discipline	Before P5
12 Evaluation discipline	Before P7
13 Documentation & ADRs	Whenever a design decision is made
14 Release process	At v0.1.0, v0.2.0, v1.0.0
15 Master checklist	Always open — it is your progress tracker
16 Portability — run on any machine	Before the first push, before every demo on a new machine

1. Engineering principles (non-negotiable)

RULE 1 — FAIL CLOSED

If a check cannot be completed (chain unreachable, missing field, parse error), the answer is **reject or defer**, never approve. This applies to every function in `gateway/stage1` and `gateway/policy`. Tests must cover the "input missing" and "dependency down" branches.

RULE 2 — MEASURED, NEVER FABRICATED

Every number in the paper, README, or a PR description comes from a file under `evaluation/results/` produced by a runner with a recorded git SHA. Hand-typed numbers are a project-ending integrity failure. If you have not measured it, write "not yet measured".

RULE 3 — NO SECRETS IN GIT, EVER

Private keys, RPC URLs with tokens, API keys: only in `.env` (ignored) or GitHub Actions secrets. `gitLeaks` runs on every commit and every PR. The keys in `.env.example` are Hardhat's public well-known test keys and must stay the only keys in the repo.

RULE 4 — DETERMINISTIC DECISION PATH

Anything that contributes to the verdict `R` must be reproducible: same inputs → same bytes. ONNX models with pinned inputs, canonical JSON, fixed seeds. The LLM never touches `R` (ADR-0002).

RULE 5 — SMALL, REVIEWED, GREEN

One logical change per PR, $\leq \sim 400$ lines of diff where possible, CI green, one approval, squash-merge. No direct pushes to `main` (branch protection enforces it).

RULE 6 — WRITE IT DOWN

Every design decision that a reviewer could question gets an ADR (`docs/adr/`). Every module has a docstring saying what it is responsible for and what it is not. Every public function has a docstring or NatSpec.

RULE 7 — PORTABLE BY DEFAULT

The whole system must start on **any** machine with Docker from a fresh clone using `make up`, and prove itself with `make smoke`. No absolute paths, no "install X first" beyond Docker, no machine-specific config outside `.env`. See section 16 and ADR-0004.

WHY SO STRICT?

Two audiences will judge this work: an evaluation panel and paper reviewers. Both look for reproducibility and honesty first. Discipline in the codebase is what makes "anyone can re-run this and get the same result" a true statement.

2. Toolchain & environment setup

Tool	Version	Purpose	Install
Python	3.11.x (pinned; 3.12 untested)	publisher, gateway, fleet, ML, eval	system <code>python3.11</code>
Node.js / npm	20 LTS	Hardhat, dashboard	system or <code>nvm</code>
Docker + compose plugin	≥ 24	hardhat node, Kubo IPFS, Ollama	system
git	≥ 2.40	—	system
syft	latest	SBOM generation (P5)	<code>curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh sh -s -- -b ~/.local/bin</code>
Ollama	latest	LLM explainer (P6)	already installed; or via compose

TWO WAYS TO RUN

Demo mode (any machine, only Docker needed): `make doctor` && `make up` && `make smoke`.

Host mode (development with hot reload): the toolchain table above plus `make bootstrap`.

Both use the same images, contracts and `.env`; the smoke test is identical.

First-time setup — host mode (P0-06)

```
cd ~/Desktop/IOT/verigate-fw
./scripts/check-env.sh           # verifies versions above
make bootstrap                   # venv + pip install -e ".[dev,eval]" + npm ci x2 + pre-commit
hooks + .env
make infra-up                    # hardhat :8545, kubo :5001, ollama :11434
docker compose -f infra/docker-compose.yml exec ollama ollama pull qwen2.5:3b-instruct # once, ~2 GB
make contracts-deploy-local      # writes contract addresses into .env
make test-all                   # must be green before you start
```

LAPTOP BUDGET

i7-1255U / 15 GB RAM / no GPU. Full stack $\approx$ 5 GB. Ollama is the heavy one (~2.5 GB) — stop it (`docker compose stop ollama`) when not working on P6, and set `LLM_ENABLED=false` in `.env`.

IDE

VS Code recommended. Install: Python, Ruff, Mypy Type Checker, Solidity (Nomic Foundation), ESLint, Prettier, EditorConfig. Set the interpreter to `./venv/bin/python`. `.vscode/` is git-ignored; keep personal settings personal.

3. Repository layout

The full layout as it exists today (placeholders will fill in as phases complete):

CHANGELOG.md	.github/workflows/release.yml
CHECKLIST.md	.github/workflows/security.yml
contracts/contracts/README.md	.gitignore
contracts/hardhat.config.ts	infra/docker-compose.yml
contracts/package.json	infra/Dockerfile.app
contracts/scripts/README.md	infra/Dockerfile.dashboard
contracts/.solhint.json	infra/Dockerfile.hardhat
contracts/test/README.md	infra/nginx.conf
contracts/tsconfig.json	Makefile
CONTRIBUTING.md	models/MANIFEST.sha256
dashboard/README.md	models/README.md
data/MANIFEST.sha256	.nvmrc
data/README.md	.pre-commit-config.yaml
data/sources.yaml	pyproject.toml
docs/adr/0000-template.md	.python-version
docs/adr/0001-sign-manifest-not-hash.md	README.md
docs/adr/0002-llm-explain-only.md	scripts/check-env.sh
docs/adr/0003-dashboard-stack.md	scripts/doctor.sh
docs/adr/0004-portable-by-default.md	scripts/README.md
docs/adr/README.md	scripts/smoke.sh
docs/diagrams/verigate_architecture.png	SECURITY.md
docs/diagrams/verigate_architecture.py	src/verigate/attacks/__init__.py
docs/diagrams/verigate_architecture.svg	src/verigate/common/__init__.py
docs/paper/.gitkeep	src/verigate/common/README.md
docs/README.md	src/verigate/fleet/__init__.py
docs/srs/.gitkeep	src/verigate/gateway/api/__init__.py
docs/threat-model.md	src/verigate/gateway/__init__.py
docs/VeriGate-FW_Developer_Manual.html	src/verigate/gateway/policy/__init__.py
docs/VeriGate-FW_Developer_Manual.pdf	src/verigate/gateway/stage1/__init__.py
docs/VeriGate-FW_Project_Guide.pdf	src/verigate/gateway/stage2/__init__.py
.editorconfig	src/verigate/gateway/verdicts/__init__.py
.env.example	src/verigate/__init__.py
evaluation/configs/.gitkeep	src/verigate/ml/features/__init__.py
evaluation/figures/.gitkeep	src/verigate/ml/__init__.py
evaluation/README.md	src/verigate/ml/train/__init__.py
evaluation/results/.gitkeep	src/verigate/ml/vulndb/__init__.py
evaluation/runners/.gitkeep	src/verigate/publisher/__init__.py
.gitattributes	src/verigate/py.typed
.github/CODEOWNERS	tests/conftest.py
.github/dependabot.yml	tests/e2e/.gitkeep
.github/ISSUE_TEMPLATE/bug_report.md	tests/fixtures/.gitkeep
.github/ISSUE_TEMPLATE/task.md	tests/integration/.gitkeep
.github/PULL_REQUEST_TEMPLATE.md	tests/README.md
.github/workflows/ci.yml	tests/unit/.gitkeep
.github/workflows/deploy-contracts.yml	.tool-versions

Placement rules — "where does this go?"

If it is...	It goes in...	Not in...
Solidity, contract tests, deploy scripts	contracts/	anywhere Python
Python used at runtime by any service	src/verigate/<area>/	scripts/, evaluation/
Code shared by two or more areas	src/verigate/common/	uplicated in each area
A pure check / scorer / transform	a function with no I/O; I/O injected via a client argument	a function that reads env or opens sockets itself
Reading environment variables	common/settings.py only	os.environ anywhere else (ruff will not catch this — reviewers will)
A test	tests/unit integration e2e/ mirroring the source path	next to the source
Test data (small)	tests/fixtures/	data/

Datasets (large, regenerable)	<code>data/raw processed/</code> (ignored) + <code>data/MANIFEST.sha256</code>	git
Trained model	<code>models/*.onnx</code> + card + MANIFEST	pickles anywhere
Experiment definition / results / figures	<code>evaluation/configs results figures/</code>	notebooks with numbers in cells
A one-off helper you ran twice	refactor into <code>src/verigate/</code> with tests	<code>scripts/</code>
A design decision	<code>docs/adr/NNNN-*.md</code>	a PR comment that gets lost
Docker / compose	<code>infra/</code>	repo root

Python package structure rules

- Single installable package `verigate` under `src/` (src-layout: tests import the installed package, never the working directory).
- Each area has `__init__.py` with a one-line docstring of responsibility; large areas have a `README.md` table of modules (see `common/README.md`).
- Dependency direction: `common` $\leftarrow$ `ml` $\leftarrow$ `gateway`; `publisher`, `fleet`, `attacks` depend on `common` only (`attacks` may import `fleet`). Never import `gateway` from `fleet` or vice-versa — they talk over HTTP.
- Entry points are declared in `pyproject.toml` (`verigate-publish`, `verigate-gateway`, `verigate-fleet`, `verigate-attack`, `verigate-train`). CLIs use Typer and live in `<area>/cli.py`.

4. Coding standards

4.1 Python

Concern	Standard	Enforced by
Formatting	ruff format (black-compatible), 100 cols	pre-commit, CI
Linting	ruff rule sets E,F,W,I,N,UP,B,A,C4,SIM,RET,ARG,PTH,S,T20,D (see <code>pyproject.toml</code>)	pre-commit, CI
Typing	mypy <code>strict</code> ; every function fully annotated; Any needs a comment	pre-commit, CI
Docstrings	Google style on every public module/class/function	ruff D rules
Data models	pydantic v2 <code>BaseModel</code> with <code>model_config = ConfigDict(extra="forbid", frozen=True)</code> for anything signed or hashed	review
Config	<code>pydantic-settings</code> in <code>common/settings.py</code> ; injected, never global-imported into pure functions	review
Logging	<code>structlog</code> ; <code>log.info("stage1.check", check="expiry", release_id=..., ok=False)</code> — event name + key/values, never f-strings. No <code>print</code> (ruff T20)	ruff, review
Errors	Domain exceptions in <code>common/errors.py</code> (<code>VerificationError</code> , <code>ChainError</code> , <code>IpfsError</code>); never bare <code>except:</code>	ruff B, review
Paths	<code>pathlib.Path</code> only (ruff PTH)	ruff
Async	Gateway API and fleet are async; ML inference and crypto are sync and called via <code>asyncio.to_thread</code> if > 10 ms	review
Security lint	ruff S (bandit) rules; <code>subprocess</code> only with list args; no <code>pickle.load</code> on untrusted data (models are ONNX, not pickle)	ruff

Example — the shape every Stage-1 check takes

```
@dataclass(frozen=True)
class CheckResult:
    name: str
    ok: bool
    reason: str | None = None

def check_version_monotonic(manifest: Manifest, device: DeviceState) -> CheckResult:
    """Reject any release whose version is not strictly greater than the installed one.

    Prevents rollback attacks (Guide §5, check #4). Pure: no I/O.
    """
    if manifest.version <= device.installed_version:
        return CheckResult("version_monotonic", False,
                           f"{manifest.version} <= installed {device.installed_version}")
    return CheckResult("version_monotonic", True)
```

4.2 Solidity

Concern	Standard
Compiler	<code>pragma solidity 0.8.26</code> ; exact; optimizer 200 runs; evm cancun (<code>hardhat.config.ts</code>)

Access control	OpenZeppelin <code>AccessControl</code> ; roles <code>ADMIN_ROLE</code> , <code>PUBLISHER_ROLE</code> , <code>GATEWAY_ROLE</code> ; never <code>tx.origin</code>
Errors	Custom errors (<code>error NotActivePublisher(bytes32 did);</code>), not revert strings — cheaper and typed
Events	Emit on every state change; indexed ids; the gateway and dashboard are event-driven
Storage	Layout documented in a comment block at the top of each contract; append-only; ADR for any change after v0.1.0
Patterns	Checks-effects-interactions; no external calls in registries anyway; no loops over unbounded arrays (batch by Merkle root instead)
Docs	NatSpec (<code>@notice</code> , <code>@param</code> , <code>@dev</code>) on every public/external item
Testing	Hardhat + chai + <code>loadFixture</code> ; every revert path tested; coverage $\geq 95\%$; gas report in PR
Static analysis	solhint (pre-commit), slither (CI security job, fails on medium+)

4.3 TypeScript / React (dashboard) and Hardhat scripts

- TypeScript `strict`; eslint (typescript-eslint recommended, react-hooks); prettier (2 spaces, semicolons, single quotes).
- Components are function components; state via zustand; server data via a typed `api/` client generated from the gateway's OpenAPI (`openapi-typescript`).
- The dashboard is **read-only on-chain** and has **no private keys**; all writes go through the gateway API (ADR-0003).

4.4 Cross-language invariants

GOLDEN VECTORS

Canonical JSON, manifest hashing, Merkle leaves and Ed25519 signatures are computed in Python (gateway) and verified in Solidity/TypeScript (contracts, dashboard). `tests/fixtures/canonical/*.json` holds input $\rightarrow$ expected bytes/hash pairs. Both the pytest suite and the Hardhat suite load the same files. If you change a hashing rule, you change the fixtures, and both suites fail until both sides agree.

5. Git workflow

5.1 Branching — trunk-based

- `main` is always releasable and protected: PR required, CI required, linear history, no force-push.
- Short-lived branches from `main`: `feat/<area>-<desc>`, `fix/...`, `docs/...`, `refactor/...`, `eval/...`, `ci/...`. Example: `feat/stage1-expiry-check`. Lifetime target: ≤ 3 days.
- Rebase on `main` before opening the PR (`git fetch && git rebase origin/main`). Squash-merge; delete branch.
- Tags: `vMAJOR.MINOR.PATCH` (SemVer). Tagging triggers the release workflow.

5.2 Commits — Conventional Commits (enforced by commit-msg hook)

```
<type>(<scope>): <imperative summary ≤ 72 chars>

[body: what and why, not how; wrap at 100]
[footer: Closes #12 · Checklist: P3-04 · BREAKING CHANGE: ...]

types : feat fix docs test refactor perf chore ci build eval
scopes: contracts publisher gateway stage1 stage2 policy verdicts fleet ml attacks dashboard infra eval docs
```

Examples: `feat(stage1): add manifest expiry check (P3-01)` · `fix(contracts): reject weights not summing to 1e4` · `eval(gas): batching vs per-device, 5 reps on sepolia`.

5.3 Pull requests

- Use the template (`.github/PULL_REQUEST_TEMPLATE.md`). Fill "How it was tested" with the actual commands and observed output.
- Reference the checklist ID and tick it in `CHECKLIST.md` in the same PR.
- Reviewer checklist: does it fail closed? are error branches tested? any env access outside settings? any number not traceable to results/? docstrings present? ADR needed?
- CI must be green. Do not merge with "will fix in follow-up" on a red check.

5.4 Solo-developer note

If you are the only developer, still open PRs and self-review the diff on GitHub the next morning. The PR record is your audit trail for the panel ("show me how the rollback check was tested" → link to PR #17).

6. Testing strategy

Layer	Where	Needs	When it runs	Target
Contract unit	<code>contracts/test/</code>	nothing (in-process EVM)	every commit (CI contracts job)	≥ 95 % lines; all reverts
Python unit	<code>tests/unit/</code>	nothing	every commit	≥ 80 % overall (enforced), 100 % for <code>stage1</code> , <code>policy</code> , <code>merkle</code> , <code>canonical</code>

Property	tests/unit/ with hypothesis	nothing	every commit	canonical JSON idempotence, Merkle proof soundness, SemVer ordering, sign/verify round-trip
Python integration	tests/ integration/	hardhat node + local IPFS backend	CI integration job, make test- integration	every contract binding, publish flow, verdict commit
End-to-end	tests/e2e/	docker compose stack	nightly, before release, make test-e2e	one test per attack scenario asserting the verdict
ML	tests/unit/ml/	fixtures	every commit	feature extraction golden vectors; ONNX output == sklearn output (atol 1e-6); model hash stable across runs

TABLE-DRIVEN TESTS FOR STAGE 1

Each check gets a parametrised test with rows (input mutation, expected ok, expected reason). Add a row for: valid · field missing · field malformed · boundary (version equal, expiry == now) · dependency unavailable.

ATTACK TESTS ARE THE DEMO

Every `src/verigate/attacks/<name>.py` has a matching `tests/e2e/test_attack_<name>.py` that runs it against a small fleet and asserts the verdict and the dashboard-visible log line. The demo script is just these in sequence.

7. CI/CD pipeline

Four GitHub Actions workflows live in `.github/workflows/`. All are already written; they will start passing as phases complete.

Workflow	Trigger	Jobs	Blocks merge?
<code>ci.yml</code>	push to main, every PR	python (ruff, ruff-format, mypy, unit + coverage) · contracts (solhint, compile, test with gas report, coverage) · integration (starts hardhat, deploys, runs integration tests with local IPFS backend, LLM disabled) · dashboard (eslint, tsc, build) · portability (compose config, builds all images, runs <code>scripts/smoke.sh</code> from a clean checkout)	Yes — all five are required checks
<code>security.yml</code>	every PR + weekly	gitleaks (secrets in history) · deps (pip-audit, npm audit high+) · slither (SARIF to code-scanning, fails on medium+)	gitleaks + slither yes; deps advisory
<code>release.yml</code>	tag <code>v*.*.*</code>	re-runs <code>ci.yml</code> → builds wheel → builds & pushes <code>ghcr.io/.../gateway:vX.Y.Z</code> → verifies <code>models/MANIFEST.sha256</code> → GitHub Release with wheel, ONNX models, manifest, auto release notes	—
<code>deploy-contracts.yml</code>	manual (<code>workflow_dispatch</code>) with typed confirmation	deploys to Arbitrum Sepolia using repository secrets in the <code>arbitrum-sepolia</code> environment (requires reviewer approval) → commits <code>deployments/arbitrumSepolia/addresses.json</code>	—

Pipeline diagram

```

developer —commit→ pre-commit hooks (ruff · mypy · gitleaks · solhint · prettier · commit-msg)
               —push→ PR → ci.yml [python || contracts || dashboard] → integration → ✓ required
                        ↳ security.yml [gitleaks || deps || slither] → review + approve → squash-merge → main
                        PR → ci.yml [portability: build images || smoke.sh] → ✓

required
main —tag vX.Y.Z→ release.yml → wheel + docker image (GHCN) + GitHub Release (models, manifest)
manual —dispatch→ deploy-contracts.yml —(environment approval)→ Arbitrum Sepolia → addresses
committed

```

Required one-time GitHub settings (P0-09, P0-10)

- Settings → Branches → add rule for `main`: require PR, require status checks (Python · lint · type · unit, Solidity · lint · test · coverage · gas, Python · integration, Dashboard · lint · type · build, Portability · compose config · image builds · fresh-clone smoke, gitleaks, slither), require linear history, include administrators.
- Settings → Environments → new `arbitrum-sepolia` → required reviewers: you → secrets `ARB_SEPOLIA_RPC_URL`, `ARB_SEPOLIA_DEPLOYER_KEY` (a throw-away key funded from a faucet, never reused).
- Settings → Code security → enable Dependabot alerts and code scanning (slither SARIF upload needs it).

4. Settings → Actions → Workflow permissions: read + write (release and deploy workflows commit/push).

When CI is red

- Reproduce locally with the same command CI ran (each step is a one-liner in the YAML). `make test-all` covers python + contracts + dashboard.
- Integration job red but local green → almost always `.env` vs `.env.example` drift, or a test that silently needs Kubo. Set `IPFS_BACKEND=local LLM_ENABLED=false` locally to match CI.
- Never mark a flaky test `skip` to get green. Fix it or open an issue and `xfail(strict=True)` with the issue link.

8. Secrets & configuration

Item	Local dev	CI	Evaluation (Sepolia)
Deployer / gateway keys	Hardhat well-known keys from <code>.env.example</code>	same	throw-away key in GitHub environment secret
Publisher Ed25519 keys	<code>keys/</code> (git-ignored), generated by <code>verigate-publish keygen</code>	generated per test run	generated per run; recorded by DID only
RPC URL	<code>http://127.0.0.1:8545</code>	same	secret (public Arbitrum Sepolia RPC or a free Alchemy/Infura key)
Contract addresses	written to <code>.env</code> by deploy script	written by deploy step	committed under <code>contracts/deployments/arbitrumSepolia/</code> (public data)
Vulnerability data	cached in <code>data/processed/vulndb</code> ; <code>VULN_CACHE_ONLY=true</code> for offline demo	fixtures only	cached, snapshot date recorded in results <code>env.json</code>

All configuration is read once in `common/settings.py` (pydantic-settings, `.env` file, env override). If a new setting is needed: add it there with a type and default, document it in `.env.example`, and pass it in — do not read `os.environ` elsewhere.

9. Daily development loop

```
git switch main && git pull
git switch -c feat/<area>-<desc>
make infra-up                                # if you need chain/ipfs
# ... write a failing test, make it pass, refactor ...
make test-unit                               # seconds
make lint typecheck                          # or rely on pre-commit
git add -p && git commit                     # hooks run; conventional message enforced
git fetch && git rebase origin/main
git push -u origin HEAD                      # open PR with template; tick CHECKLIST item in same PR
# CI green → self/peer review → squash-merge → delete branch
```

Useful targets

```
make help                list everything
make up / down / logs    whole stack in docker (demo mode); down V=1 wipes volumes
make smoke               fresh-machine proof (up → deploy → publish → verify → APPROVE → down)
make doctor              diagnose a machine: docker, RAM, disk, ports, .env, lockfiles, stray absolute
paths
make llm-pull            start ollama + pull qwen2.5:3b-instruct once
make gateway             uvicorn with reload on :8000 (docs at /docs)
make fleet N=20          20 emulated devices
make dashboard           vite on :5173
make train               retrain both models (seed 42), export ONNX, write cards
make models-hash         regenerate models/MANIFEST.sha256 (do this in the same PR as a model change)
make eval EXP=evaluation/configs/<name>.yaml
```

10. Phase-by-phase build guide

Each phase lists: goal · key design constraints · Definition of Done. Item IDs map to `CHECKLIST.md`.

P0 — Repository & tooling (week 1)

Goal: fresh clone → `make bootstrap` & `make test-all` green → first push → CI green → branch protection on.

- Bootstrap the dashboard with Vite (`npm create vite@latest . -- --template react-ts` inside `dashboard/`), add eslint + prettier, ensure `npm run lint` and `npm run build` exist.
- Commit lockfiles — CI uses `npm ci`, which requires them.
- Add a `LICENSE` (MIT recommended for a research artefact you want cited).

DoD: P0-06 ... P0-12 ticked; four workflows visible and green on `main`.

P1 — Common library & contracts (weeks 1-2)

Goal: the cryptographic and on-chain foundation, fully tested, before any service logic.

- **Canonical JSON first.** Everything signed/hashed depends on it. Write the golden vectors, then the Python, then the TypeScript helper used by Hardhat tests.
- **Manifest model:** `extra="forbid"`, `frozen=True`, SemVer-typed version, ISO-8601 UTC expiry, CIDs validated as CIDv1.
- **Contracts in dependency order** (see `contracts/contracts/README.md`). Define interfaces (`I*.sol`) first so the gateway bindings can be generated early.

- `FirmwareRegistry.register` must: check caller has `PUBLISHER_ROLE` and is `ACTIVE`; enforce version strictly greater than the publisher's last version for that `deviceModel`; store `keccak256(canonicalManifest)`; emit `NewRelease(releaseId, publisherDid, version, manifestHash)`.
- `VerdictRegistry.commitBatch(root, count, modelHash)` stores root + block + gateway; `verifyLeaf(root, leaf, proof)` view; `staleByModel(modelHash)` view returns batch ids whose model is revoked (join with `ModelRegistry`).
- `PolicyContract`: weights as basis points summing to 10 000; thresholds as basis points; `version` increments per change; event carries old/new.
- Deploy script is idempotent and writes addresses both to JSON and `.env` (localhost only).

DoD: contract coverage $\geq 95\%$, slither clean, gas report in PR, Python integration test deploys and round-trips a record, threat model table filled.

P2 — Publisher CLI (week 2)

- Typer CLI; every command prints a JSON result (machine-readable for tests and the demo script).
- `release` = hash files → put on IPFS → build manifest → sign → put manifest → `register` on-chain → print `releaseId`, CIDs, tx hash. Each step idempotent (re-running with the same inputs is safe).
- Fixtures: three versions of a toy firmware (a few hundred KB of deterministic bytes + a real ELF header) with SBOMs, so Stage 1/2 tests have realistic inputs.

DoD: integration test runs the full publish flow; `verigate-publish --help` is documented in README.

P3 — Gateway Stage 1 & fleet (weeks 3-4)

- Stage 1 checks are pure functions over (`Manifest`, `OnChainRecord`, `DeviceState`, `RevocationView`, `now`). The runner fetches inputs via injected clients, then calls the checks. First failure stops (fail-closed) but the result lists the failing check name — the dashboard shows it.
- Event listener: poll `NewRelease` from `last_processed_block` (persisted), exponential backoff on RPC errors, never skip blocks.
- Device emulator: `DeviceState` persisted as JSON under `fleet/state/<deviceId>/nvs.json` plus `slot_a.bin/slot_b.bin`. Device re-verifies hash and signature itself before switching slots. Install receipt = Ed25519 signature over canonical `{deviceId, releaseId, version, installedAt}`.
- Device ↔ gateway protocol: HTTP JSON, every message signed, nonce + timestamp window against replay.
- Attack scripts for the six Stage-1 attacks; each has an e2e test.

DoD: demo shows tamper / forge / stolen-key / rollback / freeze / sbom-swap all rejected with the right reason.

P4 — Verdicts, batching, reputation, dashboard (weeks 5-6)

- Verdict record schema (pydantic, frozen): `releaseId, deviceId, modelHashes, featureHash, r_sbom, r_img, reputation, R, verdict, rationaleCid, gatewaySig, ts`. Leaf = `SHA-256(domain || canonical(record))`.
- Batching: accumulate up to N verdicts or T seconds, build tree, `commitBatch`; store proofs locally; expose `/verdicts/{id}/proof`.

- Policy engine reads `PolicyContract` (cached per block); returns DEFER on any dependency error.
- Dashboard: pages listed in `dashboard/README.md`; live log over websocket; attack buttons.
- Tag `v0.1.0` — the crypto-path baseline that is demonstrable on its own.

P5 — SBOM risk model (weeks 7-8)

- Data: pin upstream releases in `data/sources.yaml`; fetch script; syft → CycloneDX; record every file hash in `data/MANIFEST.sha256`.
- Vulnerability clients with disk cache and a recorded snapshot date; offline mode for demos.
- Features (pure, golden-vector tested): max CVSS, sum EPSS, KEV count, n components, n outdated, mean dep age (days).
- Model: `HistGradientBoostingRegressor`; target defined in the model card (state exactly what the label is and how it was derived). Seed 42. Export ONNX; test ONNX vs sklearn equality.
- Baseline scorer (deterministic CVSS/EPSS/KEV formula) for the evaluation comparison.

DoD: model card complete, hash in MANIFEST, registered on-chain, verdicts carry it, evaluation config produces AUROC/Spearman vs baseline.

P6 — Image anomaly, LLM explainer, model revocation (weeks 9-10)

- Mutation generator with a documented catalogue (name, description, parameters, seed) — this is part of the paper's reproducibility.
- Image features (pure): chunk entropy mean/std/max, printable ratio, section sizes and deltas vs previous version, header sanity flags, appended-bytes length.
- Isolation Forest on benign releases only; evaluate per mutation class.
- Explainer: Ollama client with strict JSON schema, one retry, timeout 120 s, `LLM_ENABLED` switch; output to IPFS; CID in the verdict. Never in `R`.
- Model revocation: admin CLI → `ModelRegistry.revoke` → gateway job queries `staleByModel` → re-runs Stage 2 with successor model → new batch → dashboard shows before/after verdict delta.
- Tag `v0.2.0`.

P7 — Evaluation (weeks 11-12)

See section 12. Deploy to Sepolia via the manual workflow; record tx hashes; ≥ 5 repetitions; figures only from scripts.

P8 — Paper, demo, hand-in (weeks 13-14)

SRS finalised, paper drafted from measured results, demo rehearsed three times with a recorded fallback, `v1.0.0` released with artefacts.

11. ML & data discipline

- **Datasets are never committed.** They are reproducible from `data/sources.yaml` + fetch script + mutation generator with fixed seeds, and their hashes are in `data/MANIFEST.sha256` (committed).
- **Seeds everywhere:** numpy, sklearn, mutation generator, train/test split. `make train` twice must produce byte-identical ONNX files (test this — it is what makes model hashes meaningful).
- **Model card per model** (`models/<name>.card.md`): purpose · features (with units) · training data manifest hash · seed · hyper-parameters · metrics on held-out set · known failure modes · date · git SHA.
- **ONNX only at runtime.** No pickles loaded by the gateway. `modelHash = SHA-256(onnx bytes)` is what goes on-chain.
- **Shipping a model version:** train → card → `make models-hash` → PR (CI verifies manifest) → merge → admin registers hash on-chain → old hash remains, may be revoked later. Never overwrite a model file that has been registered; bump the file name (`image_anomaly_v2.onnx`).
- **Feature extraction is pure** and golden-vector tested; any change is a new model version because the feature hash changes meaning.
- **Honest labels.** The tampered set is synthetic; say so in the card and paper. Do not describe it as real-world malware.

12. Evaluation discipline

- One YAML per experiment in `evaluation/configs/`: seed, fleet size, network, repetitions, model hashes, git SHA is auto-recorded.
- Runner writes `evaluation/results/<exp>/<timestamp>_<sha>/raw.csv`, `summary.json` (median, IQR, n), `env.json` (CPU, RAM, Python/Node versions, contract addresses, tx hashes, vulndb snapshot date).
- Results directories are append-only. Re-running creates a new directory; nothing is edited by hand. Results that were wrong stay, with a note in the next summary.
- `evaluation/figures.py` is the only producer of figures; it reads results and writes `evaluation/figures/*.pdf|png`. Figures are regenerated in the P8 PR so they match the final results.
- Timing: ≥ 5 repetitions, warm-up run discarded, report median + IQR, state the machine.
- Gas: from hardhat-gas-reporter (local) and real receipts (Sepolia); report both and say which is which.
- Attack matrix: one row per attack, columns = expected verdict, observed verdict, run count, pass rate. Any row that is not 100 % goes into the limitations section, not the bin.

13. Documentation & ADRs

- `README.md`: what it is, quick start, layout, status link. Keep it under two screens; details go to `docs/`.
- `CHANGELOG.md`: Keep-a-Changelog format; every PR adds a line under Unreleased; release moves it under the version.
- **ADRs** (`docs/adr/`): number, title, status, context, decision, alternatives, consequences. Written when the decision is made, never edited after acceptance — superseded by a new one. Three

exist already (manifest signing, LLM explain-only, dashboard stack). Expected next: batching parameters, reputation formula, version-comparison rules, mutation catalogue.

- **Threat model** (`docs/threat-model.md`): table with asset · attacker · attack · mitigation · test that proves it. Every row links to a test file.
- API docs are generated by FastAPI at `/docs` ; contract docs by NatSpec (`npx hardhat docgen optional`).

14. Release process

1. All checklist items for the phase ticked; `main` green; `CHANGELOG.md` Unreleased section moved under the new version with date.
2. Bump `version` in `pyproject.toml`, `src/verigate/__init__.py`, `contracts/package.json`, `dashboard/package.json` in one `chore(release): vX.Y.Z` PR.
3. After merge: `git tag -a vX.Y.Z -m "..." && git push origin vX.Y.Z`. The release workflow builds and publishes.
4. Check the GitHub Release page: wheel, ONNX models, MANIFEST attached; image visible in GHCR; notes sensible.
5. Milestones: **v0.1.0** crypto path (end P4) · **v0.2.0** full AI gate (end P6) · **v1.0.0** evaluated, paper-ready (end P8). Optionally archive v1.0.0 on Zenodo for a DOI to cite in the paper.

15. Master checklist

Mirror of `CHECKLIST.md` at the time of writing. The file in the repository is authoritative — tick items there, in the same PR that completes them.

VeriGate-FW — Master Build Checklist

Single source of truth for progress. Update in the same PR that completes an item. IDs are referenced in PRs and issues. Legend: ☐ todo · ☒ in progress · ☒ done (PR merged, CI green) · ☒ dropped (say why in ADR).

Definition of Done for every item: code + tests + docstrings/NatSpec + CHANGELOG line + CI green + reviewed + squash-merged.

P0 — Repository & tooling (week 1)

- ☒ P0-01 Scaffold layout, `pyproject.toml`, `Makefile`, `.editorconfig`, `.gitignore`
- ☒ P0-02 Pre-commit: `ruff`, `mypy`, `gitleaks`, conventional commits, `solhint`, `prettier`
- ☒ P0-03 GitHub Actions: `ci.yml`, `security.yml`, `release.yml`, `deploy-contracts.yml`
- ☒ P0-04 Templates: PR, issues, CODEOWNERS, dependabot; `CONTRIBUTING.md`, `SECURITY.md`
- ☒ P0-05 ADRs 0001–0003; docs index; Project Guide and diagrams in `docs/`
- ☐ P0-06 `make bootstrap` runs clean on a fresh clone (`venv`, `npm ci` in contracts + dashboard, hooks)
- ☐ P0-07 Commit lockfiles (`contracts/package-lock.json`, `dashboard/package-lock.json`)
- ☐ P0-08 `dashboard/` bootstrapped with Vite react-ts; `npm run lint/build` pass
- ☐ P0-09 First push to GitHub; branch protection on `main` (PR required, CI required, linear history)
- ☐ P0-10 `arbitrum-sepolia` environment created with required reviewer; secrets added (RPC URL, deployer key)
- ☐ P0-11 CI green on `main` for all four workflows (security may run on schedule)
- ☐ P0-12 Choose licence (MIT recommended for a research artefact) – add `LICENSE`

P1 — Common library & contracts (weeks 1-2)

- ☐ P1-01 `common/settings.py` (pydantic-settings), `common/logging.py`
- ☐ P1-02 `common/canonical.py` + golden vectors shared with TS (`tests/fixtures/canonical/*.json`)
- ☐ P1-03 `common/crypto.py` Ed25519 + SHA-256; property tests (hypothesis)
- ☐ P1-04 `common/manifest.py` model, sign, verify; rejects unknown fields; version parsed as SemVer
- ☐ P1-05 `common/merkle.py` domain-separated leaves, proofs; cross-checked against Solidity lib
- ☐ P1-06 `PublisherRegistry.sol` + tests (register, rotate, revoke, reputation update, all revert paths)
- ☐ P1-07 `ModelRegistry.sol` + tests (register, revoke, successor, revokedAt)
- ☐ P1-08 `FirmwareRegistry.sol` + tests (ACTIVE-publisher-only, monotonic version per model, revoke, events)
- ☐ P1-09 `PolicyContract.sol` + tests (bounded weights sum to $1e4$, $\tau_{\text{approve}} < \tau_{\text{reject}}$, versioned, event)
- ☐ P1-10 `VerdictRegistry.sol` + tests (gateway-role-only, batch root, leaf proof verify, stale-by-model query)
- ☐ P1-11 `scripts/deploy.ts` idempotent; writes `deployments/<net>/addresses.json` and local `.env`
- ☐ P1-12 Contract coverage $\geq 95\%$ lines; slither: no medium+ findings; gas report committed
- ☐ P1-13 `common/chain.py` bindings from ABI artifacts; integration test: deploy $\rightarrow$ register $\rightarrow$ read
- ☐ P1-14 `common/ipfs.py` Kubo + LocalCid backends produce identical CIDv1 for fixtures
- ☐ P1-15 `docs/threat-model.md` filled from Guide §9 with "test that proves it" column

P2 — Publisher CLI (week 2)

- P2-01 `verigate-publish keygen` → keys under `keys/` (git-ignored)
- P2-02 `verigate-publish register` → `PublisherRegistry`
- P2-03 `verigate-publish release --fw --sbom --version --model --expiry` → IPFS + manifest + on-chain
- P2-04 `verigate-publish revoke <releaseId>`
- P2-05 Integration test: full publish flow against hardhat + local IPFS
- P2-06 Sample release fixtures (3 versions of a toy firmware + SBOMs) in `tests/fixtures/releases/`

P3 — Gateway Stage 1 & fleet (weeks 3-4)

- P3-01 `stage1/checks.py`: 8 checks as pure functions returning `CheckResult`; table-driven tests
- P3-02 `stage1/runner.py`: fail-closed orchestration; first failure short-circuits; structured log
- P3-03 `gateway/api`: `/health`, `/releases`, `/verify/{releaseId}`, websocket `/logs`
- P3-04 Event listener for `NewRelease` (polling with backoff; resumable from last block)
- P3-05 `fleet/device.py`: identity, NVS JSON, A/B slots, self-verify hash+sig, install receipt
- P3-06 `fleet/runner.py`: asyncio N devices; `verigate-fleet run --count N`
- P3-07 Device ↔ gateway protocol (HTTP, signed messages, nonce replay protection)
- P3-08 Attack scripts: tamper, forge, stolen-key, rollback, freeze, sbom-swap → each has an e2e test asserting REJECT/DEFER
- P3-09 Integration tests for Stage 1 against deployed contracts

P4 — Verdicts, batching, reputation, dashboard (weeks 5-6)

- P4-01 `verdicts/record.py` schema; `featureHash` = SHA-256(canonical feature vector)
- P4-02 `verdicts/batch.py` Merkle batching + `VerdictRegistry` writer; proof endpoint
- P4-03 Reputation update: receipts ↑, rejects ↓ (EWMA), written by gateway role
- P4-04 `policy/engine.py` with on-chain weights; DEFER on outage; unit tests for every branch
- P4-05 Dashboard pages: Releases, Verdicts, Fleet, Publishers, Models, Policy, Attacks
- P4-06 Dashboard live log via websocket; attack buttons call gateway `/attacks/{name}`
- P4-07 e2e: docker compose full stack, publish → verify → install → receipt → reputation
- P4-08 Demo script v1 (`scripts/demo.sh`) runs all P3 attacks end-to-end in < 5 min
- P4-09 Tag `v0.1.0` – crypto-path baseline release

P5 — SBOM risk model (weeks 7-8)

- P5-01 `ml/vulndb`: OSV, EPSS, KEV clients with disk cache; offline mode
- P5-02 `ml/data`: fetch pinned firmware releases; syft SBOMs; `data/MANIFEST.sha256`
- P5-03 `ml/features/sbom_features.py` + golden vectors
- P5-04 Training script (seed 42) → `models/sbom_risk.onnx` + card; ONNX == sklearn on test set
- P5-05 Baseline: deterministic CVSS/EPSS/KEV scorer (arXiv 2601.01308-style) for comparison
- P5-06 `stage2/sbom.py` inference; SHAP top-3; cached per release
- P5-07 Evaluation config `sbom_ranking.yaml`; AUROC / Spearman vs baseline recorded
- P5-08 Register model hash on-chain via admin CLI; verdict carries `modelHash`

P6 — Image anomaly, LLM explainer, model revocation (weeks 9-10)

- P6-01 `ml/data/mutate.py`: byte-patch, append, section-swap, pack, downgrade-relabel (documented, seeded)
- P6-02 `ml/features/image_features.py` + golden vectors
- P6-03 Isolation Forest training → `models/image_anomaly.onnx` + card

- P6-04 `stage2/image.py` inference; per-mutation-class precision/recall in evaluation
- P6-05 `stage2/explain.py`: Ollama client, strict JSON schema, retry, `LLM_ENABLED` switch; rationale → IPFS
- P6-06 Model revocation flow: revoke → stale query → re-verify job → dashboard shows delta
- P6-07 Attack scripts: vulnerable-but-genuine, hidden-payload, bad-history, poisoned-model, policy-tamper
- P6-08 Tag `v0.2.0` – full AI gate release

P7 — Evaluation (weeks 11-12)

- P7-01 Experiment configs: `latency_stage1`, `latency_stage2`, `gas_per_verdict_vs_batched`, `detection_f1`, `sbom_ranking`, `revocation_propagation`, `attack_matrix`
- P7-02 Deploy to Arbitrum Sepolia via workflow; addresses committed; tx hashes recorded
- P7-03 All runners produce `raw.csv` + `summary.json` + `env.json`; ≥ 5 repetitions
- P7-04 `evaluation/figures.py` regenerates every figure from results
- P7-05 Comparison table vs Uptane / LedgerGuard / DIDAuth-IoTFW / SBOM-triage (feature presence, cited)
- P7-06 Limitations section drafted from measured failures (what the anomaly detector missed, etc.)

P8 — Paper, demo, hand-in (weeks 13-14)

- P8-01 SRS finalised in `docs/srs/`
- P8-02 Paper draft in `docs/paper/` (LaTeX), figures from P7-04 only
- P8-03 Demo rehearsal ×3 with `scripts/demo.sh`; recorded video fallback
- P8-04 README updated with results summary and reproduction steps
- P8-05 Tag `v1.0.0`; GitHub Release with models, wheel, gas report; Zenodo DOI (optional)
- P8-06 Final `pip-audit`, `npm audit`, `slither` clean; `secrets scan` clean

16. Portability — run on any machine

The demo will happen on a computer that is not yours, and reviewers will try to reproduce the results on theirs. Portability is therefore a tested feature, not a hope. This section is the contract.

16.1 What "portable" means here

Requirement	How it is met	Proved by
Starts from a fresh clone with one command	<code>make up</code> → <code>docker compose --profile app up --build</code> ; deployer runs contracts, gateway waits on health checks, fleet waits on gateway	<code>make smoke</code> ; CI portability job on every PR
No host toolchain required for demo	All services containerised (<code>infra/</code> <code>Dockerfile.app</code> , <code>.hardhat</code> , <code>.dashboard</code>); host Python/Node only for development	CI runner has only Docker for the smoke step
Same behaviour on Linux / macOS / Windows	Pinned image tags; <code>.gitattributes</code> forces LF; no shell-specific tricks in scripts (bash, POSIX tools); Windows uses WSL2 + Docker Desktop	P8-07 fresh-machine rehearsal
No machine-specific configuration in the repo	Everything variable lives in <code>.env</code> (copied from <code>.env.example</code>); ports overridable (<code>GATEWAY_PORT</code> , <code>HARDHAT_PORT</code> , ...); paths relative to repo root or container	<code>make doctor</code> greps for absolute home paths and fails if found
Works with less RAM	LLM is an optional profile (<code>--profile llm</code>); everything else fits in ~3 GB	<code>make doctor</code> RAM check
Works offline (demo room without Wi-Fi)	<code>VULN_CACHE_ONLY=true</code> , <code>LLM_ENABLED=false</code> ; vulnerability data cached in <code>data/processed/vulndb</code> ; images pre-pulled	P8-08 offline rehearsal
Data and models reproducible elsewhere	Models committed (ONNX + MANIFEST); datasets regenerated from <code>data/sources.yaml</code> with fixed seeds; results carry <code>env.json</code>	P5/P6 determinism tests
State survives restarts, can be wiped cleanly	Named volumes (<code>ipfs-data</code> , <code>ollama-data</code> , <code>fleet-state</code>); <code>make down V=1</code> wipes	<code>smoke test runs down -v</code>

16.2 Moving to a new machine — the ritual

1. Install Docker (Desktop on macOS/Windows-WSL2; docker-ce + compose plugin on Linux). Nothing else.
2. `git clone <repo> && cd verigate-fw`
3. `make doctor` # fix every ✗ (usually: a port in use → set `GATEWAY_PORT=8001` in `.env`)
4. `make up` # first run builds images (~3–5 min); later runs are seconds
5. `make smoke` # must print "✓ smoke test passed"
6. `make llm-pull` # only if the machine has ≥ 12 GB RAM and you want the explainer live
7. open `http://localhost:5173` (dashboard) and `http://localhost:8000/docs` (gateway API)
8. `./scripts/demo.sh` # (P4-08) the rehearsed attack sequence

BEFORE A DEMO ON SOMEONE ELSE'S PC

Do steps 1–5 the day before, on that PC. Pre-pull images (`docker compose ... pull`) so no download happens live. Keep the recorded video fallback (P8-03) on a USB stick.

16.3 Rules that keep it portable (enforced in review)

- Never hard-code a path, hostname or port. Read it from `settings`; default to a container-friendly value (`0.0.0.0` , service names like `hardhat` , `ipfs`).
- Every new service gets: a pinned base image, a health check, `depends_on ... condition` , a named volume if it has state, and an entry in `.env.example` for its port.
- Anything that downloads at runtime (vulnerability data, LLM weights, firmware datasets) must have a cached/offline mode and a documented pre-fetch command.
- Scripts are `#!/usr/bin/env bash` with `set -euo pipefail` , use only POSIX tools + `docker` + `python3` , and are run by CI so they cannot rot.
- If a change breaks `make smoke` , the PR is red; fixing the smoke test is part of the change, not a follow-up.

16.4 Known platform notes

Platform	Note
Linux	Reference platform. Add your user to the <code>docker</code> group to avoid <code>sudo</code> .
macOS (Apple Silicon)	All images are multi-arch (python, node, kubo, ollama, nginx). Give Docker Desktop ≥ 6 GB memory in settings.
Windows	Use WSL2 (Ubuntu) + Docker Desktop with WSL integration. Clone inside the WSL filesystem (<code>~/</code>), not <code>/mnt/c</code> , or file watching and performance suffer. <code>make</code> is available via <code>apt install make</code> .
Low RAM (8 GB)	Skip <code>--profile llm</code> ; set <code>FLEET_SIZE=10</code> . Everything else runs.

Appendix A — Key config files (verbatim from the repository)

A.1 Makefile

```
# All day-to-day commands. `make help` lists them.
SHELL := /bin/bash
PY := .venv/bin/python
PIP := .venv/bin/pip
N ?= 20

.PHONY: up down logs smoke doctor llm-pull help bootstrap lint format typecheck test test-unit test-
integration test-contracts test-all \
    infra-up infra-down contracts-build contracts-deploy-local gateway fleet dashboard \
    train models-hash eval clean

up:                ## run EVERYTHING in docker (any machine, no host toolchain needed)
    [ -f .env ] || cp .env.example .env
    docker compose -f infra/docker-compose.yml --profile app up -d --build
    @echo "dashboard → http://localhost:${DASHBOARD_PORT:-5173}    gateway → http://localhost:${
{GATEWAY_PORT:-8000}}/docs"

down:              ## stop everything (keeps volumes; add V=1 to wipe)
    docker compose -f infra/docker-compose.yml --profile app --profile llm down $(if $(V),-v,)

logs:              ## follow all container logs
    docker compose -f infra/docker-compose.yml --profile app logs -f --tail=100

smoke:             ## fresh-machine proof: up → deploy → publish → verify → APPROVE → down
    ./scripts/smoke.sh

doctor:            ## diagnose this machine before running anything
    ./scripts/doctor.sh

llm-pull:          ## start ollama and pull the explainer model once (~2 GB)
    docker compose -f infra/docker-compose.yml --profile llm up -d ollama
    docker compose -f infra/docker-compose.yml exec ollama ollama pull qwen2.5:3b-instruct

help:              ## show this help
    @grep -E '^[a-zA-Z_-]+:.*?## ' $(MAKEFILE_LIST) | awk 'BEGIN{FS=":.*?## "} {printf "
\033[36m%-24s\033[0m %s\n", $$1, $$2}'

bootstrap:         ## create venv, install python+node deps, install git hooks
    python3.11 -m venv .venv && $(PIP) install -U pip && $(PIP) install -e ".[dev,eval]"
    cd contracts && npm ci
    cd dashboard && npm ci
    .venv/bin/pre-commit install --hook-type pre-commit --hook-type commit-msg
    [ -f .env ] || cp .env.example .env

lint:              ## ruff lint + solhint + eslint
    .venv/bin/ruff check src tests evaluation
    cd contracts && npx solhint "contracts/**/*.sol"
    cd dashboard && npm run lint

format:           ## auto-format everything
    .venv/bin/ruff format src tests evaluation && .venv/bin/ruff check --fix src tests evaluation
    cd dashboard && npx prettier --write "src/**/*.{ts,tsx,css}"

typecheck:        ## mypy strict + tsc
    .venv/bin/mypy
    cd dashboard && npx tsc --noEmit

test-unit:        ## fast python tests (no infra)
    .venv/bin/pytest -m "not integration and not e2e"

test-integration: ## python tests against hardhat + ipfs (make infra-up first)
```

```

    .venv/bin/pytest -m integration

test-contracts: ## hardhat tests + coverage
    cd contracts && npx hardhat test && npx hardhat coverage

test-all: lint typecheck test-unit test-contracts ## what CI runs

infra-up: ## start hardhat node, kubo, ollama
    docker compose -f infra/docker-compose.yml up -d

infra-down: ## stop them
    docker compose -f infra/docker-compose.yml down

contracts-build: ## compile + typechain
    cd contracts && npx hardhat compile

contracts-deploy-local: contracts-build ## deploy to local node and write addresses into .env
    cd contracts && npx hardhat run scripts/deploy.ts --network localhost

gateway: ## run the gateway API with reload
    .venv/bin/uvicorn verigate.gateway.api.main:app --reload --port 8000

fleet: ## run N emulated devices
    .venv/bin/verigate-fleet run --count $(N)

dashboard: ## vite dev server
    cd dashboard && npm run dev

train: ## train both ML models, export ONNX, write model cards
    .venv/bin/verigate-train all --seed 42

models-hash: ## regenerate models/MANIFEST.sha256
    cd models && sha256sum *.onnx > MANIFEST.sha256 && cat MANIFEST.sha256

eval: ## run an experiment: make eval EXP=evaluation/configs/gas_batching.yaml
    .venv/bin/python evaluation/runners/run.py $(EXP)

clean:
    rm -rf .venv .pytest_cache .mypy_cache .ruff_cache htmlcov contracts/artifacts contracts/cache
    dashboard/dist

```

A.2 .github/workflows/ci.yml

```

name: CI
on:
  push: { branches: [main] }
  pull_request:
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
permissions: { contents: read }

jobs:
  python:
    name: Python · lint · type · unit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11", cache: pip }
      - run: pip install -e ".[dev]"
      - run: ruff check src tests evaluation
      - run: ruff format --check src tests evaluation
      - run: mypy
      - run: pytest -m "not integration and not e2e" --cov-report=xml
      - uses: actions/upload-artifact@v4
        with: { name: coverage-python, path: coverage.xml }

  contracts:
    name: Solidity · lint · test · coverage · gas

```

```

runs-on: ubuntu-latest
defaults: { run: { working-directory: contracts } }
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: 20, cache: npm, cache-dependency-path: contracts/package-lock.json }
  - run: npm ci
  - run: npx solhint "contracts/**/*.sol"
  - run: npx hardhat compile
  - run: REPORT_GAS=true npx hardhat test
  - run: npx hardhat coverage
  - uses: actions/upload-artifact@v4
    with: { name: contracts-coverage, path: contracts/coverage }

integration:
name: Python · integration (hardhat + local IPFS backend)
runs-on: ubuntu-latest
needs: [python, contracts]
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with: { python-version: "3.11", cache: pip }
  - uses: actions/setup-node@v4
    with: { node-version: 20, cache: npm, cache-dependency-path: contracts/package-lock.json }
  - run: pip install -e ".[dev]"
  - run: cd contracts && npm ci && (npx hardhat node > /tmp/hh.log 2>&1 &) && sleep 5
  - run: cd contracts && npx hardhat run scripts/deploy.ts --network localhost
  - run: cp .env.example .env && IPFS_BACKEND=local LLM_ENABLED=false pytest -m integration

portability:
name: Portability · compose config · image builds · fresh-clone smoke
runs-on: ubuntu-latest
needs: [python, contracts]
steps:
  - uses: actions/checkout@v4
  - run: cp .env.example .env
  - run: docker compose -f infra/docker-compose.yml --profile app --profile llm config -q
  - uses: docker/setup-buildx-action@v3
  - run: docker compose -f infra/docker-compose.yml --profile app build
  - run: ./scripts/smoke.sh # up → deploy → publish fixture → verify → assert verdict →
down

dashboard:
name: Dashboard · lint · type · build
runs-on: ubuntu-latest
defaults: { run: { working-directory: dashboard } }
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: 20, cache: npm, cache-dependency-path: dashboard/package-lock.json }
  - run: npm ci
  - run: npm run lint
  - run: npx tsc --noEmit
  - run: npm run build

```

A.3 .github/workflows/security.yml

```

name: Security
on:
  pull_request:
    schedule: [{ cron: "0 3 * * 1" }] # weekly Monday 03:00 UTC
permissions: { contents: read, security-events: write }

jobs:
  secrets:
    name: gitleaks
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

```

```

- uses: gitleaks/gitleaks-action@v2
env:
  GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

deps:
  name: dependency audit
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: "3.11" }
    - run: pip install -e . pip-audit && pip-audit
    - uses: actions/setup-node@v4
      with: { node-version: 20 }
    - run: cd contracts && npm ci && npm audit --audit-level=high
    - run: cd dashboard && npm ci && npm audit --audit-level=high

slither:
  name: slither (static analysis of contracts)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: crytic/slither-action@v0.4.0
      with: { target: contracts/, sarif: slither.sarif, fail-on: medium }
    - uses: github/codeql-action/upload-sarif@v3
      if: always()
      with: { sarif_file: slither.sarif }

```

A.4 .github/workflows/release.yml

```

name: Release
on:
  push:
    tags: ["v*.*.*"]
permissions: { contents: write, packages: write }

jobs:
  verify:
    uses: ../github/workflows/ci.yml

  build-and-publish:
    needs: verify
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install build && python -m build
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v6
        with:
          context: .
          file: infra/Dockerfile.gateway
          push: true
          tags: ghcr.io/${ github.repository }/gateway:${ github.ref_name }
      - name: Verify model manifest
        run: cd models && sha256sum -c MANIFEST.sha256
      - uses: softprops/action-gh-release@v2
        with:
          generate_release_notes: true
          files: |
            dist/*
            models/*.onnx
            models/MANIFEST.sha256

```

A.5 .github/workflows/deploy-contracts.yml

```
name: Deploy contracts (Arbitrum Sepolia)
# Manual, gated. Used only for evaluation runs. Secrets are repository secrets, never in code.
on:
  workflow_dispatch:
    inputs:
      confirm:
        description: 'Type "arbitrum-sepolia" to confirm'
        required: true
permissions: { contents: write }

jobs:
  deploy:
    if: github.event.inputs.confirm == 'arbitrum-sepolia'
    runs-on: ubuntu-latest
    environment: arbitrum-sepolia # requires reviewer approval in repo settings
    defaults: { run: { working-directory: contracts } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm, cache-dependency-path: contracts/package-lock.json }
      - run: npm ci && npx hardhat compile
      - run: npx hardhat run scripts/deploy.ts --network arbitrumSepolia
      env:
        ARB_SEPOLIA_RPC_URL: ${ secrets.ARB_SEPOLIA_RPC_URL }
        ARB_SEPOLIA_DEPLOYER_KEY: ${ secrets.ARB_SEPOLIA_DEPLOYER_KEY }
      - name: Commit deployment addresses
        run: |
          git config user.name "github-actions" && git config user.email "actions@github.com"
          git add deployments/arbitrumSepolia && git commit -m "chore(contracts): record arbitrum-sepolia deployment" && git push
```

A.6 .pre-commit-config.yaml

```
# Install once: `pre-commit install --hook-type pre-commit --hook-type commit-msg`
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: end-of-file-fixer
      - id: trailing-whitespace
      - id: check-yaml
      - id: check-json
      - id: check-added-large-files
        args: ["--maxkb=2048"] # models/*.onnx are allowed up to 2 MB; datasets are git-ignored
      - id: check-merge-conflict
      - id: detect-private-key
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks:
      - id: ruff
        args: ["--fix"]
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.11.2
    hooks:
      - id: mypy
        additional_dependencies: ["pydantic>=2.8", "types-requests"]
        args: ["--config-file=pyproject.toml"]
        pass_filenames: false
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.4
    hooks:
      - id: gitleaks
  - repo: https://github.com/compilerla/conventional-pre-commit
    rev: v3.4.0
    hooks:
      - id: conventional-pre-commit
```

```

    stages: [commit-msg]
    args: ["feat", "fix", "docs", "test", "refactor", "perf", "chore", "ci", "build", "eval"]
- repo: local
  hooks:
    - id: solhint
      name: solhint
      entry: bash -c 'cd contracts && npx solhint "contracts/**/*.sol"'
      language: system
      files: \.sol$
      pass_filenames: false
    - id: prettier-dashboard
      name: prettier (dashboard)
      entry: bash -c 'cd dashboard && npx prettier --check "src/**/*.{ts,tsx,css}"'
      language: system
      files: ^dashboard/src/
      pass_filenames: false

```

A.7 .env.example

```

# Copy to .env – never commit .env. Values here are safe defaults for local dev only.
VERIGATE_ENV=local
# Blockchain
RPC_URL=http://127.0.0.1:8545
CHAIN_ID=31337
# Hardhat default account #0 – PUBLIC test key, local node only. NEVER put a real key here.
DEPLOYER_PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d96d4c9b98f
GATEWAY_PRIVATE_KEY=0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
# Contract addresses are written here by `make contracts-deploy-local`
PUBLISHER_REGISTRY_ADDR=
FIRMWARE_REGISTRY_ADDR=
MODEL_REGISTRY_ADDR=
POLICY_CONTRACT_ADDR=
VERDICT_REGISTRY_ADDR=
# IPFS
IPFS_API=http://127.0.0.1:5001
IPFS_BACKEND=kubo # kubo | local (local = in-repo CID store, same CIDs)
# LLM explainer
OLLAMA_URL=http://127.0.0.1:11434
LLM_MODEL=qwen2.5:3b-instruct
LLM_ENABLED=true
# Vulnerability data (cached under data/processed/vulndb)
OSV_API=https://api.osv.dev/v1
VULN_CACHE_ONLY=false # true = offline demo mode
# Ports (override if something on this machine already uses them)
HARDHAT_PORT=8545
IPFS_API_PORT=5001
IPFS_GATEWAY_PORT=8080
OLLAMA_PORT=11434
GATEWAY_PORT=8000
DASHBOARD_PORT=5173
FLEET_SIZE=20
# Gateway
GATEWAY_HOST=0.0.0.0
LOG_LEVEL=INFO
# Arbitrum Sepolia (evaluation only) – set via GitHub secrets / local .env, never here
# ARB_SEPOLIA_RPC_URL=
# ARB_SEPOLIA_DEPLOYER_KEY=

```

A.8 .github/PULL_REQUEST_TEMPLATE.md

```

## What
<!-- one paragraph: what this PR changes and why -->

## Checklist item
<!-- e.g. P3-04 Stage-1 expiry check -->

## How it was tested
<!-- commands run, what you observed. Paste measured numbers if any (never estimated). -->

```

```
## Checklist
- [ ] `make test-all` passes locally
- [ ] New code has tests; coverage did not drop
- [ ] Docstrings / NatSpec on every public function
- [ ] No secrets, no datasets, no hand-edited results in the diff
- [ ] `CHANGELOG.md` updated under *Unreleased*
- [ ] If a design decision was made: ADR added in `docs/adr/`
- [ ] If a contract changed: storage layout unchanged or migration noted; gas report attached
- [ ] If a model changed: `models/MANIFEST.sha256` regenerated and model card updated
```

A.9 docs/adr/0000-template.md

```
# NNNN – Title

**Status:** Proposed | Accepted | Superseded by NNNN
**Date:** YYYY-MM-DD
**Deciders:**

## Context
What problem or question forced this decision? Include constraints (laptop, time, research goals).

## Decision
What we chose, stated in one or two sentences.

## Alternatives considered
- Option A – why not
- Option B – why not

## Consequences
Positive, negative, and what becomes easier/harder. Follow-up tasks.
```

A.10 infra/docker-compose.yml

```
# ONE file, two profiles.
# docker compose -f infra/docker-compose.yml up -d          → infra only (hardhat, ipfs, ollama)
# docker compose -f infra/docker-compose.yml --profile app up -d → everything: + deployer, gateway, fleet, dashboard
# Every image is pinned. Every port is overridable via .env. No host paths other than the repo itself.
name: verigate

x-python-app: &python-app
  build: { context: .., dockerfile: infra/Dockerfile.app }
  env_file: ../.env
  environment:
    RPC_URL: http://hardhat:8545
    IPFS_API: http://ipfs:5001
    OLLAMA_URL: http://ollama:11434
  volumes:
    - ../contracts/deployments:/app/contracts/deployments:ro # addresses written by deployer
    - fleet-state:/app/fleet-state

services:
  hardhat:
    build: { context: .., dockerfile: infra/Dockerfile.hardhat }
    ports: [ "${HARDHAT_PORT:-8545}:8545" ]
    healthcheck:
      test: ["CMD", "curl", "-sf", "-X", "POST", "-H", "content-type: application/json", "-d", '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}', "http://localhost:8545"]
      interval: 5s
      timeout: 3s
      retries: 20

  ipfs:
    image: ipfs/kubo:v0.30.0
    ports: [ "${IPFS_API_PORT:-5001}:5001", "${IPFS_GATEWAY_PORT:-8080}:8080" ]
    volumes: [ "ipfs-data:/data/ipfs" ]
    environment: { IPFS_PROFILE: server }
```

```

healthcheck:
  test: ["CMD", "ipfs", "id"]
  interval: 5s
  timeout: 3s
  retries: 20

ollama:
  image: ollama/ollama:0.3.12
  ports: ["${OLLAMA_PORT:-11434}:11434"]
  volumes: ["ollama-data:/root/.ollama"]
  deploy: { resources: { limits: { memory: 4g } } }
  profiles: ["llm"]          # optional: docker compose --profile llm up (pull model once: make llm-pull)

  deployer:                  # one-shot: compiles + deploys contracts, writes deployments/localhost/addresses.json
    build: { context: .., dockerfile: infra/Dockerfile.hardhat }
    command: ["npm", "hardhat", "run", "scripts/deploy.ts", "--network", "docker"]
    environment: { RPC_URL: http://hardhat:8545 }
    volumes: [ "../contracts/deployments:/app/contracts/deployments" ]
    depends_on: { hardhat: { condition: service_healthy } }
    profiles: ["app"]

gateway:
  <<: *python-app
  command: ["verigate-gateway"]
  ports: ["${GATEWAY_PORT:-8000}:8000"]
  depends_on:
    hardhat: { condition: service_healthy }
    ipfs: { condition: service_healthy }
    deployer: { condition: service_completed_successfully }
  healthcheck:
    test: ["CMD", "python", "-c", "import urllib.request as u; u.urlopen('http://localhost:8000/health')"]
    interval: 5s
    timeout: 3s
    retries: 30
    profiles: ["app"]

fleet:
  <<: *python-app
  command: ["verigate-fleet", "run", "--count", "${FLEET_SIZE:-20}", "--gateway", "http://gateway:8000", "--state-dir", "/app/fleet-state"]
  depends_on: { gateway: { condition: service_healthy } }
  profiles: ["app"]

dashboard:
  build: { context: .., dockerfile: infra/Dockerfile.dashboard, args: { VITE_GATEWAY_URL: "http://localhost:${GATEWAY_PORT:-8000}" } }
  ports: ["${DASHBOARD_PORT:-5173}:80"]
  depends_on: { gateway: { condition: service_healthy } }
  profiles: ["app"]

volumes:
  ipfs-data:
  ollama-data:
  fleet-state:

```

A.11 infra/Dockerfile.app

```

# One image for every Python service (gateway, fleet, publisher, attacks). Compose picks the command.
FROM python:3.11.9-slim AS build
WORKDIR /app
COPY pyproject.toml README.md ./
COPY src ./src
RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir .

FROM python:3.11.9-slim
RUN useradd -m app && mkdir -p /app/fleet-state /app/contracts/deployments && chown -R app /app
COPY --from=build /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages

```

```

COPY --from=build /usr/local/bin/verigate-* /usr/local/bin/
COPY --chown=app models /app/models
COPY --chown=app tests/fixtures /app/fixtures
WORKDIR /app
USER app
EXPOSE 8000
CMD ["verigate-gateway"]

```

A.12 scripts/smoke.sh

```

#!/usr/bin/env bash
# Fresh-machine smoke test. Proves the whole stack starts from a clean clone with ONE command and
# reaches a verdict.
# Used by CI (portability job), by `make smoke`, and by you on any new laptop before a demo.
set -euo pipefail
cd "$(dirname "$0")/.."
COMPOSE="docker compose -f infra/docker-compose.yml --profile app"
cleanup() { $COMPOSE down -v --remove-orphans >/dev/null 2>&1 || true; }
trap cleanup EXIT

[ -f .env ] || cp .env.example .env
echo "► starting full stack"
$COMPOSE up -d --build
echo "► waiting for gateway health"
for i in $(seq 1 60); do
    if curl -sf "http://localhost:${GATEWAY_PORT:-8000}/health" >/dev/null; then break; fi
    sleep 2; [ "$i" -eq 60 ] && { echo "gateway never became healthy"; $COMPOSE logs --tail=50; exit 1; }
done
echo "► publishing fixture release v1.0.0 and verifying"
$COMPOSE run --rm -T gateway verigate-publish release \
    --fw /app/fixtures/releases/v1.0.0/firmware.bin --sbom /app/fixtures/releases/v1.0.0/sbom.json \
    --version 1.0.0 --model demo-device --expiry 2030-01-01T00:00:00Z --json > /tmp/release.json
RELEASE_ID=$(python3 -c 'import json,sys; print(json.load(open("/tmp/release.json"))["releaseId"])')
VERDICT=$(curl -sf -X POST "http://localhost:${GATEWAY_PORT:-8000}/verify/${RELEASE_ID}" | python3 -c
'import json,sys; print(json.load(sys.stdin)["verdict"])')
echo "verdict: $VERDICT"
[ "$VERDICT" = "APPROVE" ] || { echo "expected APPROVE for a clean fixture"; exit 1; }
echo "✓ smoke test passed"

```

A.13 scripts/doctor.sh

```

#!/usr/bin/env bash
# `make doctor` – one-screen diagnosis on ANY machine: versions, ports, docker, disk, RAM, .env,
# lockfiles.
set -uo pipefail
cd "$(dirname "$0")/.."
ok() { printf " \033[32m✓\033[0m %s\n" "$1"; }
bad() { printf " \033[31m✗\033[0m %s\n" "$1"; FAIL=1; }
FAIL=0
echo "Toolchain"
command -v docker >/dev/null && docker compose version >/dev/null 2>&1 && ok "docker + compose plugin"
|| bad "docker compose missing (Docker Desktop on macOS/Windows-WSL2, docker-ce on Linux)"
command -v python3.11 >/dev/null && ok "python3.11" || bad "python3.11 missing (only needed for host-
mode dev; docker mode does not need it)"
command -v node >/dev/null && [ "$(node -p 'process.versions.node.split(".")[0]')" -ge 20 ] && ok "node
≥ 20" || bad "node ≥ 20 missing (host-mode dev only)"
echo "Repository"
[ -f .env ] && ok ".env present" || bad ".env missing → cp .env.example .env"
[ -f contracts/package-lock.json ] && ok "contracts lockfile" || bad "contracts/package-lock.json
missing (P0-07)"
[ -f dashboard/package-lock.json ] && ok "dashboard lockfile" || bad "dashboard/package-lock.json
missing (P0-07)"
grep -rIl --exclude-dir=.git --exclude-dir=node_modules --exclude-dir=.venv "/home/[a-z]*/" . 2>/dev/
null | grep -v '^./docs/' | head -3 | while read -r f; do bad "absolute home path in $f"; done
echo "Machine"
MEM=$(awk '/MemTotal/ {printf "%d", $2/1024/1024}' /proc/meminfo 2>/dev/null || sysctl -n hw.memsize
2>/dev/null | awk '{printf "%d", $1/1024/1024}')
[ "${MEM:-0}" -ge 8 ] && ok "RAM ${MEM} GB (≥ 8 needed; 12+ with LLM)" || bad "RAM ${MEM} GB – run
without --profile llm"

```

```

DISK=$(df -Pk . | awk 'NR==2 {printf "%d", $4/1024/1024}')
[ "$DISK" -ge 10 ] && ok "disk ${DISK} GB free" || bad "disk ${DISK} GB free (< 10 GB)"
echo "Ports"
for p in ${HARDHAT_PORT:-8545} ${IPFS_API_PORT:-5001} ${GATEWAY_PORT:-8000} ${DASHBOARD_PORT:-5173}; do
  (command -v ss >/dev/null && ss -ltn | grep -q ":$p ") && bad "port $p in use → override in .env
  (e.g. GATEWAY_PORT=8001)" || ok "port $p free"
done
[ "$FAIL" -eq 0 ] && echo "all good – run: make up" || { echo "fix the ✖ items above"; exit 1; }

```

A.14 docs/adr/0004-portable-by-default.md

```

# 0004 – Portable by default: one command runs the whole stack on any machine

**Status:** Accepted    **Date:** 2026-09-18

## Context
The project must be demonstrated on hardware we do not control (panel room PC, a reviewer's laptop, a
fresh VM) and
reproduced by others for the paper. "Works on the author's laptop" is the most common FYP demo failure.

## Decision
- Every runtime component has a pinned Docker image (`infra/Dockerfile.app`, `.hardhat`, `.dashboard`;
  pinned base tags).
- `infra/docker-compose.yml` runs the full stack with `--profile app`; the LLM is an optional `--
  profile llm`.
- `make up`, `make smoke`, `make doctor` are the only commands a new machine needs; host Python/Node
  are for development only.
- No absolute paths in code or config; everything resolves relative to the repository root or a
  container path.
  Ports are overridable via `.env`. Line endings normalised via `.gitattributes`.
- CI has a `portability` job that builds all images and runs `scripts/smoke.sh` from a clean checkout
  on every PR.
- Datasets are regenerable (`data/sources.yaml` + scripts); models are committed; results carry
  `env.json`.

## Alternatives considered
- Host-only setup with a long README – rejected; drifts immediately.
- Kubernetes / Helm – rejected; overkill for a single-machine demo.

## Consequences
Image builds add ~3 min to CI. Windows users need WSL2 + Docker Desktop (documented). The smoke test
doubles as the
first e2e test and as the pre-demo ritual on any new machine.

```

End of manual. Next deliverable: [docs/srs/](#) — Software Requirements Specification.